

```

const express = require("express");
const app = express();
app.use(express.json());

// ——— DEPENDENCIAS ———
const { Redis } = require("@upstash/redis");
const crypto = require("crypto");

// ——— VARIABLES DE ENTORNO ———
const ANTHROPIC_API_KEY = process.env.ANTHROPIC_API_KEY;
const MANYCHAT_API_KEY = process.env.MANYCHAT_API_KEY;
const META_DATASET_ID = process.env.META_DATASET_ID;
const META_ACCESS_TOKEN = process.env.META_ACCESS_TOKEN;
const ALERTA_SUBSCRIBER_ID = process.env.ALERTA_SUBSCRIBER_ID;
const UPSTASH_REDIS_REST_URL = process.env.UPSTASH_REDIS_REST_URL;
const UPSTASH_REDIS_REST_TOKEN = process.env.UPSTASH_REDIS_REST_TOKEN;

// ——— REDIS ———
const redis = new Redis({
  url: UPSTASH_REDIS_REST_URL,
  token: UPSTASH_REDIS_REST_TOKEN,
});

// ——— COLA ANTI-SPAM ———
const procesando = new Set();

// ——— INFO DEL PROYECTO ———
const INFO_ENCINO = `
PRIVADA ENCINO - INFORMACION OFICIAL

```

Proyecto campestre en Area de La Morita, Montemorelos, NL.

Frente al Restaurant El Pariente. 5 min de Pueblo Salvaje, 3 min del Rio Blanquil
 Maps: <https://maps.app.goo.gl/y9ske7rVR2nBSS8s9>

Caracteristicas: unico proyecto pavimentado en la zona, acceso controlado, electr

Lotes disponibles:

- Lote 1: 1,648 m2 (38x38m) - \$1,600,000 - 24 MSI
- Lote 3: 1,639 m2 (43x38m) - \$1,726,200 - 24 MSI
- Lote 3B: 1,700 m2 (43x39m) - \$1,785,000 - 24 MSI
- Lote 4 PREMIUM: 1,632 m2 (45x38m) - \$1,600,000 - Contado - en colina con la mej

Financiamiento directo sin banco: Enganche \$350,000 mas 24 mensualidades de \$75,0
 Proceso: apartar, contrato notaria, escrituras listas.

Visitas: sabados y domingos.

`;

// — SYSTEM PROMPT —————

const SYSTEM_PROMPT = `Eres Daniel Soliz, asesor de ventas de Privada Encino en M

\${INFO_ENCINO}

PERSONALIDAD:

Profesional, directo y cordial. Estilo Monterrey. Mensajes cortos, maximo 3 linea

REGLA ABSOLUTA - INFORMACION:

SOLO usa la informacion que esta escrita arriba. NUNCA inventes cosas que no este

CITAS Y VISITAS - MUY IMPORTANTE:

Si el cliente menciona querer visitar, agendar cita, conocer el terreno, ir a ver

CALIFICACION POR PRESUPUESTO:

- Si menciona contado o pago completo: enfoca en Lote 4 PREMIUM primero
- Si menciona mensualidades o msi: enfoca en financiamiento directo sin banco
- Si pregunta "algo mas barato": "El precio inicial es desde \$1,600,000 con finan

COMO RESPONDER:

1. LEE el historial completo antes de responder. Nunca trates un mensaje como si
2. LEE el primer mensaje con atencion – si dice "precios" responde precios, si di
3. RESPONDE siempre lo que pregunto el cliente aunque escriba mal. "ubaicon"=ubic
4. NUNCA digas "no entiendo". Siempre responde algo util.
5. UNA sola pregunta por mensaje.
6. Precios: siempre "desde \$1,600,000".
7. Cuando piden fotos: "Con gusto, en un momento le comparto algunas fotos del pr

REGLA DE ORO - NUNCA DES TODO JUNTO:

Si el cliente pide precios, ubicacion, medidas y financiamiento en el mismo mensa

MENSAJES EN 2 PARTES:

Cuando necesites dar informacion Y hacer una pregunta, separa con ---

Ejemplo:

Estamos en Montemorelos, 45 min de Monterrey. Aqui el mapa: <https://maps.app.goo>.

Para que esta buscando el terreno, construccion, inversion o descanso?

FLUJO:

- Primer mensaje con intencion clara (precios/ubicacion/fotos): responde directo
- Primer mensaje sin intencion: saluda brevemente y pregunta para que busca el te
- Mensajes siguientes: continua naturalmente sin resetear.
- Objetivo: agendar visita sabado o domingo.

HORARIO: L-V 9am-9pm, S-D tambien. Fuera de horario: "Gracias por escribir, con g

SENALES - escribelas en linea separada al final, el cliente NUNCA las ve:

FOTO_ENCINO: cuando pide fotos generales

FOTO_UBICACION: cuando pide fotos de ubicacion o acceso

FOTO_LOTES: cuando pide fotos de los lotes especificos

ETIQUETA:nuevo-lead: primer mensaje

ETIQUETA:intencion-conocida: cuando dice para que busca

ETIQUETA:calificado: cuando tiene plazo definido

ETIQUETA:visita-agendada: cuando confirma visita

ALERTA_VISITA_PENDIENTE:[detalle]: quiere visitar

ALERTA_VISITA_CONFIRMADA:[nombre] el [dia]: visita confirmada

ALERTA_VISITA_OTRO_DIA:[dia]: quiere visitar dia diferente

ALERTA_AUDIO: mando audio

ALERTA_NO_SABE: no sabes responder`;

// — HELPERS —————

```
function hashSHA256(valor) {  
  if (!valor) return null;  
  return crypto.createHash("sha256").update(valor.toLowerCase().trim()).digest("h  
}
```

```
function dentroDeHorario() {  
  const ahora = new Date();  
  const hora = ahora.getHours();  
  return hora >= 9 && hora < 21;  
}
```

```
async function getConversacion(clave) {  
  try {  
    const data = await redis.get("conv:" + clave);  
    return data ? JSON.parse(data) : [];  
  } catch (e) {  
    console.error("Error Redis get:", e);  
    return [];  
  }  
}
```

```
async function setConversacion(clave, mensajes) {  
  try {  
    // TTL de 24 horas - conversacion se resetea suavemente  
    await redis.setex("conv:" + clave, 86400, JSON.stringify(mensajes));  
  } catch (e) {  
    console.error("Error Redis set:", e);  
  }  
}
```

```

async function getBotCongelado(clave) {
  try {
    const val = await redis.get("congelado:" + clave);
    return val === "true";
  } catch (e) {
    return false;
  }
}

async function setBotCongelado(clave, valor) {
  try {
    if (valor) {
      await redis.setex("congelado:" + clave, 86400, "true");
    } else {
      await redis.del("congelado:" + clave);
    }
  } catch (e) {
    console.error("Error Redis congelado:", e);
  }
}

async function esNuevoLead(clave) {
  try {
    const val = await redis.get("lead:" + clave);
    if (!val) {
      await redis.setex("lead:" + clave, 2592000, "true"); // 30 dias
      return true;
    }
    return false;
  } catch (e) {
    return false;
  }
}

// — VISITAS —————
async function guardarVisita(clave, detalle) {
  try {
    const visita = {
      clave,
      detalle,
      timestamp: Date.now(),
      recordatorio1enviado: false,
      recordatorio2enviado: false
    };
    await redis.setex("visita:" + clave, 604800, JSON.stringify(visita)); // 7 di
  } catch (e) {
    console.error("Error guardar visita:", e);
  }
}

```

```

    }
}

// ——— MANYCHAT —————
async function mandarAlerta(mensaje) {
  try {
    const response = await fetch("https://api.manychat.com/fb/sending/sendContent
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": "Bearer " + MANYCHAT_API_KEY
      },
      body: JSON.stringify({
        subscriber_id: ALERTA_SUBSCRIBER_ID,
        data: {
          version: "v2",
          content: {
            messages: [{ type: "text", text: mensaje }]
          }
        }
      })
  });
  const data = await response.json();
  console.log("ALERTA RESPONSE:", JSON.stringify(data));
} catch (e) {
  console.error("Error alerta:", e);
}
}

async function ponerEtiqueta(subscriberId, etiqueta) {
  try {
    await fetch("https://api.manychat.com/fb/subscriber/addTag", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": "Bearer " + MANYCHAT_API_KEY
      },
      body: JSON.stringify({
        subscriber_id: subscriberId,
        tag_name: etiqueta
      })
    });
  } catch (e) {
    console.error("Error etiqueta:", e);
  }
}
}

```

```

async function activarFlow(subscriberId, flowNs) {
  try {
    await fetch("https://api.manychat.com/fb/sending/sendFlow", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": "Bearer " + MANYCHAT_API_KEY
      },
      body: JSON.stringify({
        subscriber_id: subscriberId,
        flow_ns: flowNs
      })
    });
  } catch (e) {
    console.error("Error activar flow:", e);
  }
}

// — META EVENTOS —————
async function mandarEventoMeta(evento, telefono) {
  try {
    const telefonoHash = hashSHA256(telefono);
    if (!telefonoHash) return;

    await fetch("https://graph.facebook.com/v18.0/" + META_DATASET_ID + "/events"
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        data: [{
          event_name: evento,
          event_time: Math.floor(Date.now() / 1000),
          action_source: "other",
          user_data: { ph: [telefonoHash] }
        }],
        access_token: META_ACCESS_TOKEN
      })
    );
  } catch (e) {
    console.error("Error Meta:", e);
  }
}

// — SEGUIMIENTO AUTOMATICO —————
async function verificarSeguimientos() {
  try {
    const claves = await redis.keys("seguimiento:*");
    const ahora = Date.now();
  }
}

```

```

for (const clave of claves) {
  const data = await redis.get(clave);
  if (!data) continue;
  const seg = JSON.parse(data);

  // 48 horas sin respuesta - mandar alerta
  if (!seg.alertaEnviada && ahora - seg.timestamp > 172800000) {
    await mandarAlerta("Lead sin respuesta 48hrs\nSubscriber: " + seg.subscri
    seg.alertaEnviada = true;
    await redis.setex(clave, 604800, JSON.stringify(seg));
  }
}

// Reactivacion leads frios 7 dias
const clavesLeads = await redis.keys("frio:*");
for (const clave of clavesLeads) {
  const data = await redis.get(clave);
  if (!data) continue;
  const lead = JSON.parse(data);
  if (!lead.alertaEnviada && ahora - lead.timestamp > 604800000) {
    await mandarAlerta("Lead frio 7 dias\nSubscriber: " + lead.subscriberId +
    lead.alertaEnviada = true;
    await redis.setex(clave, 604800, JSON.stringify(lead));
  }
}
} catch (e) {
  console.error("Error seguimientos:", e);
}
}

// ——— REPORTE DIARIO —————
async function reporteDiario() {
  try {
    const leads = await redis.keys("lead:*");
    const visitas = await redis.keys("visita:*");
    const congelados = await redis.keys("congelado:*");
    const fecha = new Date().toLocaleDateString("es-MX");

    await mandarAlerta(
      "Reporte diario Privada Encino\n" +
      fecha + "\n\n" +
      "Leads totales: " + leads.length + "\n" +
      "Visitas pendientes: " + visitas.length + "\n" +
      "Conversaciones activas: " + congelados.length
    );
  } catch (e) {

```

```

        console.error("Error reporte:", e);
    }
}

// — SCHEDULER —————
setInterval(verificarSeguimientos, 3600000); // cada hora

// Reporte diario a las 9pm
setInterval(() => {
    const ahora = new Date();
    if (ahora.getHours() === 21 && ahora.getMinutes() < 5) {
        reporteDiario();
    }
}, 300000); // cada 5 minutos verifica si son las 9pm

// — WEBHOOK PRINCIPAL —————
app.post("/webhook", async (req, res) => {
    try {
        const { telefono, mensaje, subscriber_id, primer_mensaje } = req.body;

        console.log("BODY COMPLETO:", JSON.stringify(req.body));

        if (!mensaje) {
            return res.status(400).json({ error: "Falta mensaje" });
        }

        const clave = subscriber_id || telefono || "desconocido";

        // Anti-spam — si ya está procesando este cliente espera
        if (procesando.has(clave)) {
            await new Promise(r => setTimeout(r, 3000));
        }
        procesando.add(clave);

        try {
            // Verificar si el bot está congelado para este cliente
            const congelado = await getBotCongelado(clave);
            if (congelado) {
                console.log("Bot congelado para:", clave);
                procesando.delete(clave);
                return res.json({ respuesta1: null, respuesta2: null, alerta: "congelado" });
            }

            // Fuera de horario
            if (!dentroDeHorario()) {
                procesando.delete(clave);
                return res.json({

```



```

        respuesta1: "Gracias por escribir, con gusto le atiendo manana a primer
        respuesta2: null,
        alerta: null,
        foto: false
    });
}

// Lead nuevo
const esNuevo = await esNuevoLead(clave);
if (esNuevo) {
    await mandarEventoMeta("Lead", telefono || subscriber_id || "desconocido"
    // Guardar para seguimiento
    await redis.setex("seguimiento:" + clave, 604800, JSON.stringify({
        subscriberId: subscriber_id,
        timestamp: Date.now(),
        ultimoMensaje: mensaje,
        alertaEnviada: false
    }));
    // Guardar para reactivacion
    await redis.setex("frio:" + clave, 1209600, JSON.stringify({
        subscriberId: subscriber_id,
        timestamp: Date.now(),
        alertaEnviada: false
    }));
} else {
    // Actualizar ultimo mensaje en seguimiento
    const segData = await redis.get("seguimiento:" + clave);
    if (segData) {
        const seg = JSON.parse(segData);
        seg.ultimoMensaje = mensaje;
        seg.timestamp = Date.now();
        seg.alertaEnviada = false;
        await redis.setex("seguimiento:" + clave, 604800, JSON.stringify(seg));
    }
}

// Obtener historial
let conversacion = await getConversacion(clave);

// Si hay primer mensaje del anuncio y es nueva conversacion - agregar cont
if (primer_mensaje && conversacion.length === 0) {
    conversacion.push({
        role: "user",
        content: "[El cliente llego por un anuncio y su primer mensaje fue: " +
    });
} else {
    conversacion.push({ role: "user", content: mensaje });
}

```

```

}

// Llamar a Claude
const response = await fetch("https://api.anthropic.com/v1/messages", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "x-api-key": ANTHROPIC_API_KEY,
    "anthropic-version": "2023-06-01"
  },
  body: JSON.stringify({
    model: "claude-sonnet-4-5",
    max_tokens: 500,
    system: SYSTEM_PROMPT,
    messages: conversacion
  })
});

const data = await response.json();

if (!data.content || !data.content[0] || !data.content[0].text) {
  console.error("Error Claude:", JSON.stringify(data));
  procesando.delete(clave);
  return res.json({ respuesta1: "Un momento, dejame verificarlo.", respuesta
}

let respuesta = data.content[0].text;
console.log("RESPUESTA:", respuesta);

// Guardar en historial
conversacion.push({ role: "assistant", content: respuesta });
if (conversacion.length > 20) {
  conversacion = conversacion.slice(-20);
}
await setConversacion(clave, conversacion);

// — PROCESAR SEÑALES —————
let alerta = null;
let foto = false;
let flowFotos = null;

// Fotos
if (respuesta.includes("FOTO_ENCINO")) {
  foto = true;
  flowFotos = "FLOW_FOTOS_PROYECTO"; // reemplaza con tu flow_ns real
  respuesta = respuesta.replace(/FOTO_ENCINO/g, "").trim();
} else if (respuesta.includes("FOTO_UBICACION")) {

```

```

    foto = true;
    flowFotos = "FLOW_FOTOS_UBICACION"; // reemplaza con tu flow_ns real
    respuesta = respuesta.replace(/FOTO_UBICACION/g, "").trim();
} else if (respuesta.includes("FOTO_LOTES")) {
    foto = true;
    flowFotos = "FLOW_FOTOS_LOTES"; // reemplaza con tu flow_ns real
    respuesta = respuesta.replace(/FOTO_LOTES/g, "").trim();
}

// Etiquetas
const etiquetasMatch = respuesta.match(/ETIQUETA:[a-zA-Z0-9_-]+/g);
if (etiquetasMatch) {
    for (const e of etiquetasMatch) {
        const nombre = e.replace("ETIQUETA:", "");
        if (subscriber_id) await ponerEtiqueta(subscriber_id, nombre);
        if (nombre === "calificado") await mandarEventoMeta("CompleteRegistrati
    }
    respuesta = respuesta.replace(/ETIQUETA:[a-zA-Z0-9_-]+/g, "").trim();
}

// Alertas de visita
if (respuesta.includes("ALERTA_VISITA_PENDIENTE")) {
    const match = respuesta.match(/ALERTA_VISITA_PENDIENTE:(.+)/);
    alerta = "ALERTA_VISITA_PENDIENTE";
    respuesta = respuesta.replace(/ALERTA_VISITA_PENDIENTE:./g, "").trim();
    const detalle = match ? match[1] : "";
    await mandarAlerta("VISITA PENDIENTE\nCliente: " + (telefono || subscribe
    await guardarVisita(clave, detalle);
    // Congelar bot – tú tomas el control
    await setBotCongelado(clave, true);
    await mandarEventoMeta("InitiateCheckout", telefono || subscriber_id);
} else if (respuesta.includes("ALERTA_VISITA_CONFIRMADA")) {
    const match = respuesta.match(/ALERTA_VISITA_CONFIRMADA:(.+)/);
    alerta = "ALERTA_VISITA_CONFIRMADA";
    respuesta = respuesta.replace(/ALERTA_VISITA_CONFIRMADA:./g, "").trim();
    await mandarAlerta("VISITA CONFIRMADA\n" + (match ? match[1] : telefono))
    await mandarEventoMeta("Schedule", telefono || subscriber_id);
    if (subscriber_id) await ponerEtiqueta(subscriber_id, "visita-agendada");
} else if (respuesta.includes("ALERTA_VISITA_OTRO_DIA")) {
    const match = respuesta.match(/ALERTA_VISITA_OTRO_DIA:(.+)/);
    alerta = "ALERTA_VISITA_OTRO_DIA";
    respuesta = respuesta.replace(/ALERTA_VISITA_OTRO_DIA:./g, "").trim();
    await mandarAlerta("Visita otro dia\nCliente: " + (telefono || subscriber
} else if (respuesta.includes("ALERTA_AUDIO")) {

```

```

    alerta = "ALERTA_AUDIO";
    respuesta = respuesta.replace(/ALERTA_AUDIO/g, "").trim();
    await mandarAlerta("AUDIO recibido\nCliente: " + (telefono || subscriber_
    await setBotCongelado(clave, true);

} else if (respuesta.includes("ALERTA_NO_SABE")) {
    alerta = "ALERTA_NO_SABE";
    respuesta = respuesta.replace(/ALERTA_NO_SABE/g, "").trim();
    await mandarAlerta("No sabe responder\nCliente: " + (telefono || subscrib
}

// Activar flow de fotos si aplica
if (flowFotos && subscriber_id) {
    await activarFlow(subscriber_id, flowFotos);
}

// Separar respuesta en 2 partes
const partes = respuesta.split("---");
const respuesta1 = partes[0].trim();
const respuesta2 = partes[1] ? partes[1].trim() : null;

procesando.delete(clave);
res.json({ respuesta1, respuesta2, alerta: alerta || null, foto });

} catch (innerError) {
    procesando.delete(clave);
    throw innerError;
}

} catch (error) {
    console.error("Error webhook:", error);
    res.status(500).json({ error: "Error interno" });
}
});

// ——— DESCONGELAR BOT —————
app.post("/descongelar", async (req, res) => {
    try {
        const { clave } = req.body;
        if (!clave) return res.status(400).json({ error: "Falta clave" });
        await setBotCongelado(clave, false);
        res.json({ status: "Bot descongelado para: " + clave });
    } catch (e) {
        res.status(500).json({ error: e.message });
    }
});

```

```

// — HEALTH CHECK —————
app.get("/health", async (req, res) => {
  try {
    await redis.ping();
    res.json({
      status: "ok",
      redis: "conectado",
      hora: new Date().toLocaleTimeString("es-MX"),
      dentroHorario: dentroDeHorario()
    });
  } catch (e) {
    res.status(500).json({ status: "error", redis: "desconectado" });
  }
});

// — REPORTE MANUAL —————
app.get("/reporte", async (req, res) => {
  await reporteDiario();
  res.json({ status: "Reporte enviado" });
});

// — LIMPIAR HISTORIAL —————
app.get("/limpiar", async (req, res) => {
  try {
    const claves = await redis.keys("conv:*");
    for (const c of claves) await redis.del(c);
    res.json({ status: "Historial limpiado", conversaciones_borradas: claves.leng
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});

// — STATUS —————
app.get("/", (req, res) => {
  res.json({ status: "Agente Daniel - Privada Encino v2.0 funcionando" });
});

// — SERVIDOR —————
const PORT = process.env.PORT || 3000;
app.listen(PORT, function () {
  console.log("Servidor Daniel v2.0 corriendo en puerto " + PORT);
});

```